

ET AI HACKATHON 2026

AI-Driven Cyber Resilience for Critical National Infrastructure

Detailed Project Document — Problem Statement #7

Behavioral Anomaly Detection Engine + MITRE ATT&CK Attribution Agent

Participant: Fida M S

Submission type: Solo build

Repository: github.com/fidashameer/cyber-resilience-ai

This document is a detailed technical account of what was built, what was validated, and — deliberately — what was not built and why. Throughout, results are reported honestly rather than optimistically: where a number is weaker than a headline figure, or a capability is incomplete, that is stated directly rather than smoothed over.

1. Problem Context

Critical national infrastructure has become a primary target for sophisticated cyber actors, and India's public institutions have borne the brunt of this over the past two years. CERT-In reported handling over 1.59 million cybersecurity incidents in 2023, a figure that continued climbing through 2024-25. The pattern of high-profile attacks is concerning and recurring: AIIMS Delhi was paralysed for over two weeks by ransomware in 2022; CBSE suffered a data breach affecting examination records in 2024; and in early 2026 a coordinated cyberattack targeted CBSE's digital infrastructure ahead of board examinations, compromising student data and forcing emergency shutdowns across multiple states.

India's National Cyber Security Policy acknowledges that over 70% of government entities operate on end-of-life IT infrastructure — attackers are not working hard to find entry points. The deeper problem is detection speed: most public-sector organisations discover breaches only after significant damage has occurred, weeks or months after initial infiltration, because detection relies on signature matching, and advanced persistent threats (APTs) deliberately operate low-and-slow to stay under signature thresholds. What's needed is a behavioural intelligence layer that flags deviation from how a system normally behaves, not whether it matches a known-bad signature.

2. Challenge Statement & Scope Decision

The problem statement suggests five possible sub-agents: a Behavioural Anomaly Detection Engine, an APT Campaign Attribution & Prediction Agent, an Autonomous Incident Response Orchestrator (SOAR), a Government Infrastructure Vulnerability Prioritisation agent, and a Cyber Resilience Digital Twin.

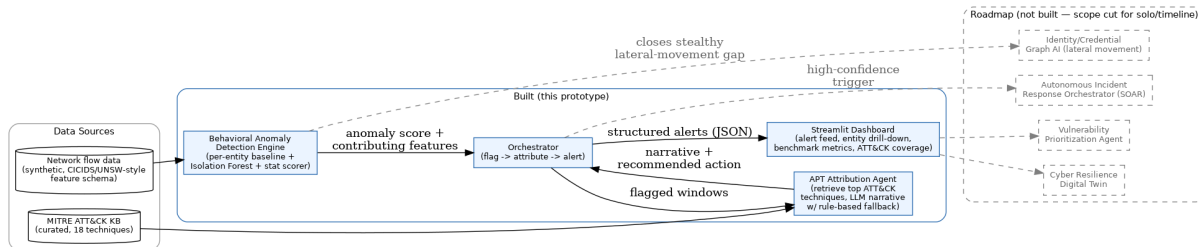
This solo build scoped deliberately to **two** of the five: the detection engine and the attribution agent. The other three were cut, not half-built — each needs live infrastructure (real SOC tooling, a live CVE/asset feed, a real organisation's security architecture) that a solo hackathon entry cannot credibly fake in the time available. The reasoning: a smaller thing that is actually true and independently validated beats a bigger thing that is asserted but not demonstrated.

Sub-agent coverage

Problem statement sub-agent	Status
Behavioural Anomaly Detection Engine	Built and benchmarked
APT Campaign Attribution and Prediction Agent	Built (attribution); next-stage move prediction is roadmap
Autonomous Incident Response Orchestrator (SOAR)	Not built — roadmap
Government Infrastructure Vulnerability Prioritisation	Not built — roadmap
Cyber Resilience Digital Twin	Not built — roadmap

3. System Architecture

Solid boxes below represent components built and working in this prototype. Dashed boxes are the roadmap items described above — shown for completeness, not implemented.



4. Component 1 — Behavioural Anomaly Detection Engine

Learns what is normal for each individual host, then flags deviation from that host's own baseline — not a match against a known signature. Four building blocks:

1. **Per-entity baseline** — mean and standard deviation of every behavioural feature, computed independently per host, fully unsupervised (no labels needed in production).
2. **Isolation Forest** — catches multi-feature combinations that are jointly unusual, even when no single feature looks extreme in isolation.
3. **Statistical max-deviation scorer** — a fast-path catch for “loud” single-feature spikes; alone it reaches 100% recall on obvious attacks. (An earlier design used Local Outlier Factor instead of this scorer; LOF was measured and found to score worse than random — 0.34 ROC-AUC alone — against these clustered synthetic anomalies, and was removed.)
4. **Peer-novelty signal (`novel_peer_ratio`), added during this build** — tracks, per entity, which destination hosts it has ever actually contacted before (causally, using only windows strictly prior to the current one, so there is no label leakage), and flags when it reaches somewhere new. This was added specifically to fix a measured weakness: stealthy lateral movement using volume-based features alone reached only 5.9% recall, because credential-based lateral movement over already-common protocols (SMB/RDP) does not necessarily move enough traffic volume to look anomalous. Reaching a never-before-contacted host is a sparser, volume-independent signal. Adding it raised stealthy lateral-movement recall to 31.0% — a real, roughly 5x improvement, though still the weakest category, stated honestly rather than rounded up.

5. Component 2 — APT Campaign Attribution Agent

A retrieve-then-generate pipeline that turns a raw anomaly score into an analyst-readable explanation with a recommended action, against a hand-curated knowledge base of 18 MITRE ATT&CK techniques across 8 tactics.

- **Retrieve** — contributing features from the detection engine are matched against each technique's documented signal features, weighted by deviation rank, to produce a ranked list of candidate techniques.

- **Generate** — an analyst-readable narrative and recommended action are synthesized from the top candidates. If an ANTHROPIC_API_KEY is set, Claude generates the narrative; if not, an automatic rule-based fallback runs instead — the pipeline never fails or blocks on this being absent.
- **Structured output** — every alert is a structured JSON record: entity, contributing features, candidate techniques with scores, generated narrative, recommended action, and (in the evaluation runs) the known ground-truth label — giving a full audit trail for every alert the system raises.

6. Results — Synthetic Evaluation Set

7,200 five-minute behavioural windows across 60 entities, roughly 410 attack windows (5.7% base rate), an even split of “loud” (obvious) and “stealthy” (low-and-slow, damped toward the entity’s own baseline) variants across 5 attack families: port scan, brute force, DoS, exfiltration, lateral movement. “Noisy normal” benign windows (legitimate large transfers, admin maintenance, scanner-like sweeps) were included deliberately so the evaluation is not artificially easy.

Metric	Value
ROC-AUC	0.990
PR-AUC	0.896
Precision @ operating threshold	0.787
Recall @ operating threshold	0.829
False positive rate	1.35%

The blended recall number hides the real story. Unblended by attack type and severity:

Attack type	Loud recall	Stealthy recall
Port scan	100%	97.6%
Brute force	97.7%	37.8%
DoS	100%	68.4%
Exfiltration	100%	64.9%
Lateral movement	100%	31.0%

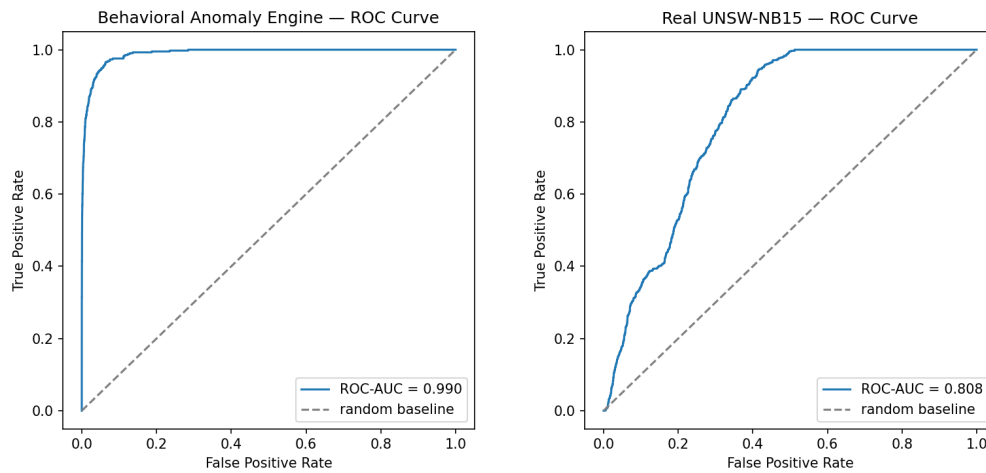
Stealthy brute-force recall fell from 67.6% to 37.8% after the peer-novelty feature was added — a genuine trade-off, not an oversight. The engine flags a fixed number of windows per run (a contamination budget); once peer-novelty made lateral-movement and port-scan windows rank higher, some borderline brute-force windows that used to just clear the cutoff no longer do, at the same total alert volume. Net effect across all attack types is still positive, but this is a real illustration of a fixed-threshold detector: improving one axis can cost recall on another sharing the same alert budget. Per-attack-type adaptive thresholds (roadmap) would remove this trade-off.

7. Independent Validation — Real UNSW-NB15 Data

Everything above was necessarily tuned on synthetic data. To get an independent read on whether the approach actually generalises, the real UNSW-NB15 dataset (2.54 million raw network connections, real IPs/ports/timestamps) was obtained separately and run through the same, unmodified anomaly engine — the raw connections were aggregated into the same per-entity, 5-minute-window schema as the synthetic data (6,257 real windows, 43 real hosts, 10.5% attack-labeled).

Metric	Synthetic	Real UNSW-NB15
ROC-AUC	0.990	0.808
PR-AUC	0.896	0.251
Precision @ operating point	0.787	0.312
Recall @ operating point	0.829	0.312

The gap is real and reported directly rather than explained away. The real dataset's dominant attack categories are Exploits and Fuzzers — single malicious packets or malformed payloads, not sustained volume shifts. This engine is behavioural: built to catch sustained pattern change (scans, floods, brute force, exfiltration transfers). A one-packet exploit does not necessarily move aggregate flow statistics the way a sustained brute-force or DoS attempt does. This is a real limitation of the method, not just the dataset, and is exactly the kind of gap a payload/signature-assisted layer working alongside the behavioural one would close (roadmap). Still clearly and meaningfully above random — 0.81 ROC-AUC, curve well above the diagonal.



ROC curve — synthetic (left) vs. real UNSW-NB15 (right)

8. Attribution Accuracy at MITRE ATT&CK Technique Level

This problem statement's own stated Evaluation Focus names “APT attribution accuracy at MITRE ATT&CK technique level” directly. That is worth measuring honestly rather than asserting the attribution agent “works” from a couple of example outputs — so an explicit evaluation script (`attribution_eval.py`) was built for it, producing two metrics at two different levels of rigor.

Technique-level accuracy — 96.2% top-1, 98.8% top-3

The obvious objection to any “attribution accuracy” claim is that no dataset has ground-truth ATT&CK technique-ID labels — that normally requires a red team tagging every packet with the exact technique used. That objection is valid for third-party data (real UNSW-NB15, where we did not choose what technique each connection represents), but it does **not** hold for the synthetic data: `generate_dataset.py`'s attack simulators were explicitly written to model specific named techniques (port scan → T1595 Active Scanning, brute force → T1110 Brute Force, DoS → T1498 Network Denial of Service, exfiltration → T1041 Exfiltration Over C2 Channel, lateral movement → T1021 Remote Services). That design intent is a legitimate ground truth for data generated by the same simulator, so exact technique-ID accuracy is measured directly against it — not asserted, not proxied. Where the simulator does not distinguish sibling sub-techniques (e.g. password-guessing vs. password-spraying under brute force), the expected label is the parent technique, and a retrieved sub-technique under that same parent counts as correct, since claiming more precision than the simulator actually has would be dishonest in the other direction.

Attack type	Expected technique	Technique top-1	Technique top-3
Port scan	T1595	100%	100%
DoS	T1498	97.3%	100%
Brute force	T1110	96.4%	100%
Lateral movement	T1021	94.9%	96.6%
Exfiltration	T1041	91.4%	97.1%

Tactic-level accuracy — 97.4% top-1, 99.4% top-3

A looser, complementary check reported alongside the technique-level number rather than instead of it: does the top candidate's ATT&CK *tactic* match what is expected for the known attack type (e.g. a port scan should retrieve something tagged Reconnaissance/Discovery, not Credential Access).

Both metrics are measured only on the synthetic dataset. Real UNSW-NB15's own categories (Exploits, Fuzzers, Generic, ...) are not ATT&CK labels at all and do not map onto the curated 18-technique knowledge base cleanly enough to make either check meaningful there — forcing that mapping would produce a number that looks precise without being honest. $n = 340$ true-positive alerts evaluated in both cases.

9. Honesty Notes & Known Limitations

This build ran inside a sandboxed environment with an outbound network allowlist limited to PyPI, GitHub, and the Anthropic API. Kaggle, UCI, HuggingFace, GitHub raw content, and attack.mitre.org were all initially unreachable, which is why the primary evaluation dataset is synthetic rather than a pre-existing benchmark file, and why real UNSW-NB15 validation was added afterward once the user supplied the raw data directly.

The MITRE ATT&CK knowledge base is hand-curated (18 techniques across 8 tactics), not a live TAXII/STIX feed. Technique IDs and descriptions are accurate to the public framework, but this is a snapshot, not a live feed — production deployment should swap this for the `mitreattack-python` library against the official TAXII server.

A handful of real-data features (`internal_dst_ratio`, `admin_proto_ratio`, `failed_auth_count`) are necessarily approximations given what UNSW-NB15's raw connection records actually log — documented explicitly in `load_unsw_real.py`'s header comment.

Scalability — an honest, currently untested area

This prototype was evaluated at 7,200 synthetic and 6,257 real windows on a single machine — small relative to a real critical-infrastructure deployment monitoring thousands of hosts continuously. The architecture has a scalability-friendly shape: per-entity baselines are computed independently, so baseline fitting and scoring can be sharded by entity without cross-entity coordination; the detection engine is stateless per scoring call given a baseline, so scoring can run behind a horizontally-scaled service; and the attribution agent's retrieval step is a fixed lookup against 18 curated techniques, an near-constant cost per alert regardless of alert volume. What is **not** yet built or tested: streaming ingestion (the current pipeline is batch/CSV), a distributed baseline store for cross-node consistency, and load testing at real production volumes with sub-second scoring SLAs. These are concrete next steps to go from “architecture that scales in principle” to a load-tested claim — stated here as an open gap rather than asserted as already solved.

10. Roadmap — What Closes the Remaining Gaps

- 1. Identity/credential-graph signals** — Closes the rest of the stealthy lateral-movement gap. Peer-novelty closed part of it using traffic data alone; the honest ceiling for a flow-only signal is well below what real auth/identity data (which account authenticated to which host) would achieve.
- 2. Per-attack-type adaptive thresholds** — Removes the brute-force-vs-lateral-movement trade-off in Section 6 — one global alert budget is the root cause.
- 3. Payload/signature-assisted layer** — Closes the real-data gap in Section 7. Exploit/fuzzing attacks need packet-content inspection alongside behavioural baselining.
- 4. Next-stage move prediction** — The attribution agent currently explains what already happened; ranking likely next ATT&CK techniques given the current campaign stage was in-scope for this sub-agent originally.
- 5. Autonomous Incident Response (SOAR)** — Pre-approved containment playbooks (isolate endpoint, revoke credential, block IP), gated by confidence threshold, with human escalation above a blast-radius limit.
- 6. Streaming ingestion and load testing** — Moves the architecture from batch/CSV to real-time streaming and validates the scalability claims in Section 9 against production-scale volumes.

11. Running the Prototype

Full setup instructions, dependencies, and repository layout are documented in the repository README. In brief:

```
cd src
pip install -r ../requirements.txt

python generate_dataset.py # writes data/network_windows.csv
python evaluate.py # writes eval/metrics.json, roc_curve.png, pr_curve.png
python orchestrator.py # writes eval/alerts.json, run_summary.json

streamlit run dashboard.py # opens the demo UI
```

12. Links

Repository: <https://github.com/fidashameer/cyber-resilience-ai>

Full README (problem, methodology, all results, honesty notes): in repository root

Architecture diagram, presentation deck, demo video: submitted alongside this document per hackathon deliverable requirements